

Integrating symbolic execution with LLMs for automated generation of C program specifications

Fanpeng Yang, Xu Ma, Shuling Wang*, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu

Abstract—Automatically generating formal specifications including loop invariants, preconditions, and postconditions for legacy code is critical for program understanding, reuse, and verification. However, the inherent complexity of control and data structures in programs makes this task particularly challenging, especially when no verification goal is provided to drive synthesis. This paper presents SESPEC, a framework that integrates symbolic execution with large language models (LLMs) and a formal verifier (Frama-C/WP) to synthesize verifier-checked C program specifications. For loop-free fragments fully covered by the symbolic executor, SESPEC computes exact path-sensitive postconditions, composing the specifications of ordinary (non-recursive) callees along the way; at the remaining boundaries (loops, recursive data structures, and fragments outside the symbolic executor), it uses the available symbolic facts to guide LLM synthesis whose candidate specifications are then checked by the verifier. During symbolic execution, SESPEC decomposes the synthesis task by pausing at program boundaries, where the symbolic state is injected into annotation templates and LLM prompts. When a specification is faulty, verifier feedback is combined with symbolic information to refine the failed candidate rather than re-sampling blindly. This design supports goal-independent specification generation, because it derives functional relations from symbolic states and verifier feedback rather than relying solely on externally provided assertions. Through extensive evaluation across numerical, pointer, struct, and linked-list benchmark families and an additional goal-independent benchmark (SESPEC-400), SESPEC outperforms the state-of-the-art LLM-based tools across diverse program classes, including the goal-independent setting in which no verification target is provided.

Index Terms—Program specification generation, loop invariants, symbolic execution, large language models, formal verification.

I. INTRODUCTION

Across software domains, from general-purpose systems to long-lived industrial control codebases, a vast body of legacy code lacks formal specifications, i.e., precise, verifiable descriptions of program behavior such as preconditions, postconditions, and loop invariants. Automating specification generation is therefore essential for program comprehension,

reuse, and verification. The task is challenging due to the interplay of complex control structures (nested branches, loops, function calls) and data representations (structs, pointers, inductive datatypes). In this work, we tackle *goal-independent* functional specification generation: inferring verifier-checked preconditions, postconditions, and loop invariants that capture functional behavior without requiring a user-supplied assertion as the synthesis target. The goal is not to prove one preselected property, but to recover verifier-checked specifications that capture a program’s functional behavior, whether or not symbolic execution can derive them exactly.

Existing work on program specification generation has largely focused on loop invariants. Traditional symbolic methods such as constraint solving [1], [2], Craig interpolation [3]–[5], recurrence analysis [6], [7], shape analysis [8], and dynamic methods such as Daikon [9] are each restricted to particular invariant forms or program classes; recent ML-augmented or SMT-augmented methods [10]–[12] are similarly confined to numeric programs. LLM-based approaches have begun to broaden the program coverage, but tend to produce specifications that are either ill-formed, too weak, or both, and recent integrations with static/formal techniques [13]–[16] only partially address this. AUTOSPEC [13] and Preguss [16] are goal-dependent (requiring a predefined verification target or RTE target) and degrade on complex data structures; SPEC-GEN [15] is goal-independent but only in the Java/JML setting, and “verifier-accepted” alone says nothing about strength. These limitations leave open the problem of generating goal-independent, semantically informative, verifier-checked specifications for C programs with loops, calls, pointers, structs, and inductive data.

To address this open problem, we propose SESPEC, a framework that integrates symbolic execution, LLM-based synthesis, and formal verification. The key idea is to let symbolic execution drive the process. The analysis proceeds through the program and pauses at program boundaries (loops, calls, and recursive constructs), turning one whole-function specification task into a sequence of smaller local synthesis problems. On loop-free segments fully handled by the flattened-memory symbolic executor, symbolic execution computes an exact path-sensitive postcondition, which is the strongest postcondition within that model; outside this envelope, symbolic execution provides partial semantic anchors for LLM synthesis. At each boundary, the symbolic state, i.e. the symbolic store of variable values, the path conditions, and the types of the in-scope variables, collectively the *local symbolic context*, is injected into invariant templates and LLM prompts. When a candidate fails, formal verification feedback

*Corresponding author: Shuling Wang (e-mail: wangsl@ios.ac.cn).

F. Yang, X. Ma, S. Wang, and X. Xu are with the Institute of Software, Chinese Academy of Sciences, and also with the University of Chinese Academy of Sciences, Beijing, China (e-mail: yangfp@ios.ac.cn; maxu@ios.ac.cn; wangsl@ios.ac.cn; xux@ios.ac.cn).

Q. Cao is with the School of Computer Science and School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China (e-mail: cao-qinxiang@sjtu.edu.cn).

N. Zhan is with the School of Computer Science, Peking University, Beijing, China (e-mail: znj@ios.ac.cn).

X. Li and B. Gu are with the Beijing Institute of Control Engineering, Beijing, China (e-mail: 1351127502@126.com; gubin@ios.ac.cn).

is combined with symbolic information to refine the failed candidate rather than re-sampling blindly.

We implement SESPEC on top of QCP’s symbolic execution engine [17], [18] and Frama-C/WP formal verifier [19]. We perform a comprehensive evaluation of SESPEC, including an ablation study that confirms the contribution of each component. We evaluate on benchmark families from prior invariant- and specification-generation work, covering numerical loops, multi-/nested loops, pointers, structs, and singly linked lists. On numerical benchmarks, SESPEC approaches the performance of tools specialized for numerical programs and consistently outperforms recent LLM-based approaches such as AUTOSPEC [13] and ACInv [14] (a static-analysis-guided LLM invariant generator). On the pointer-, struct-, and linked-list-heavy families that prior LLM-based tools handle poorly, it outperforms AUTOSPEC by a clear margin. We further construct **SESPEC-400**, a 400-program benchmark with all assertions removed, to stress-test the goal-independent setting at scale, and use it to analyze the validity, strength, failure modes, and cost of generated specifications; here SESPEC substantially outperforms the single-shot LLM and AUTOSPEC baselines, and the dimension-wise comparison finds that its specifications provide more informative postconditions and stronger loop invariants. Together these results show that the symbolic-execution-guided design generalizes from the goal-dependent to the goal-independent setting without specialization.

In summary, the key contributions of this work are:

- **Symbolic-execution-guided specification synthesis:** We present a framework that uses path-sensitive symbolic states to guide LLM-based specification generation. On segments fully covered by symbolic execution, these states provide exact semantic summaries; outside the fully covered fragment, the same symbolic state still supplies local symbolic context to anchor templates and prompts for LLM synthesis.
- **Verifier-guided iterative refinement:** We design a multi-layer refinement mechanism that classifies Frama-C/WP errors into diagnostic categories and applies targeted repair strategies, progressively guiding the LLM toward verifier-checked specifications instead of re-sampling blindly.
- **Goal-independent evaluation:** We construct **SESPEC-400**, a benchmark of 400 C programs with assertions removed, and use it to conduct what we believe is the first empirical study of goal-independent specification generation, evaluating validity, strength, failure modes, and per-program specification generation cost.

The rest of the paper is organized as follows. Section II introduces a motivating example and gives an overview of our approach. Section III and Section IV present the core techniques for pre-/postcondition specification generation and loop-invariant generation, respectively. Section IV-C describes the verifier-guided refinement procedure, and Appendix D in the supplementary material presents the system design and implementation. Section V reports the empirical evaluation under established benchmark protocols. Section VI presents a goal-independent study on the **SESPEC-400** benchmark. Ap-

pendix I in the supplementary material presents ten representative case studies. Appendix J in the supplementary material discusses limitations and open challenges, and Appendix K in the supplementary material discusses threats to validity. Section VII surveys related work, and Section VIII concludes. Finally, the data-availability statement provides links to the datasets, implementation code, LLM configurations, and supplementary results.

II. A MOTIVATING EXAMPLE AND OUR APPROACH

Automating the generation of formal specifications for C code remains challenging due to the prevalence of complex language features such as loops, pointer manipulation, and data structures. Consider the code fragment shown in Fig. 1, which is adapted from a navigation component in an industrial control codebase. The code defines a struct `CheckCal` with three fields: an integer array `pkv[10]`, an integer field `len` representing the number of valid elements in the array, and an integer field `chksum` storing the sum of the array’s elements. Function `foo` then invokes `CheckCalFun` on a `CheckCal` pointer `pIp`, with a verification goal annotated by `/*@ assert... */`. The callee function `CheckCalFun` takes a `CheckCal` pointer `pIp`, computes the sum of the `len` elements in the array `pkv` via a loop, and stores the result in the field `chksum` of `pIp`. This example exhibits diverse features: complex data structures (structs, arrays and pointers), a loop with inductive operations over arrays, and pointer-induced aliasing and side effects. Consequently, it faces the following challenges:

- **Loop reasoning:** The summation loop `for(; i<pIp->len; i++)` requires determining whether the initial state allows loop entry and, if the loop executes, establishing invariants strong enough to relate the partial sum to the processed prefix of the array.
- **Structs and Pointers:** The procedures access arrays and read and write struct fields through pointers, making it essential to account for aliasing and side effects in their specifications. For instance, the specification for function `CheckCalFun` must guarantee memory safety for dereferencing `pIp->pkv[i]`, precisely describe the write effect on `pIp->chksum`, and specify frame properties for fields of `pIp` that remain unchanged.
- **Inductive predicates:** Properties of arrays in loops, such as aggregate sums, are often most naturally expressed with inductive predicates or logic functions. Automatically writing such definitions in a verifier-friendly form and then proving the resulting annotations remains a central challenge for programs involving recursive operations or data structures.

As shown in Fig. 1, the code is annotated with the full ACSL specifications enclosed in `/*@... */`, including preconditions (beginning with `requires`), postconditions (beginning with `ensures`), loop invariants (beginning with `loop invariant`), and verification goals (beginning with `assert`). Among these, the loop invariants, the default preconditions, and the postconditions are generated automatically by our tool SESPEC, while the `assert` clause represents

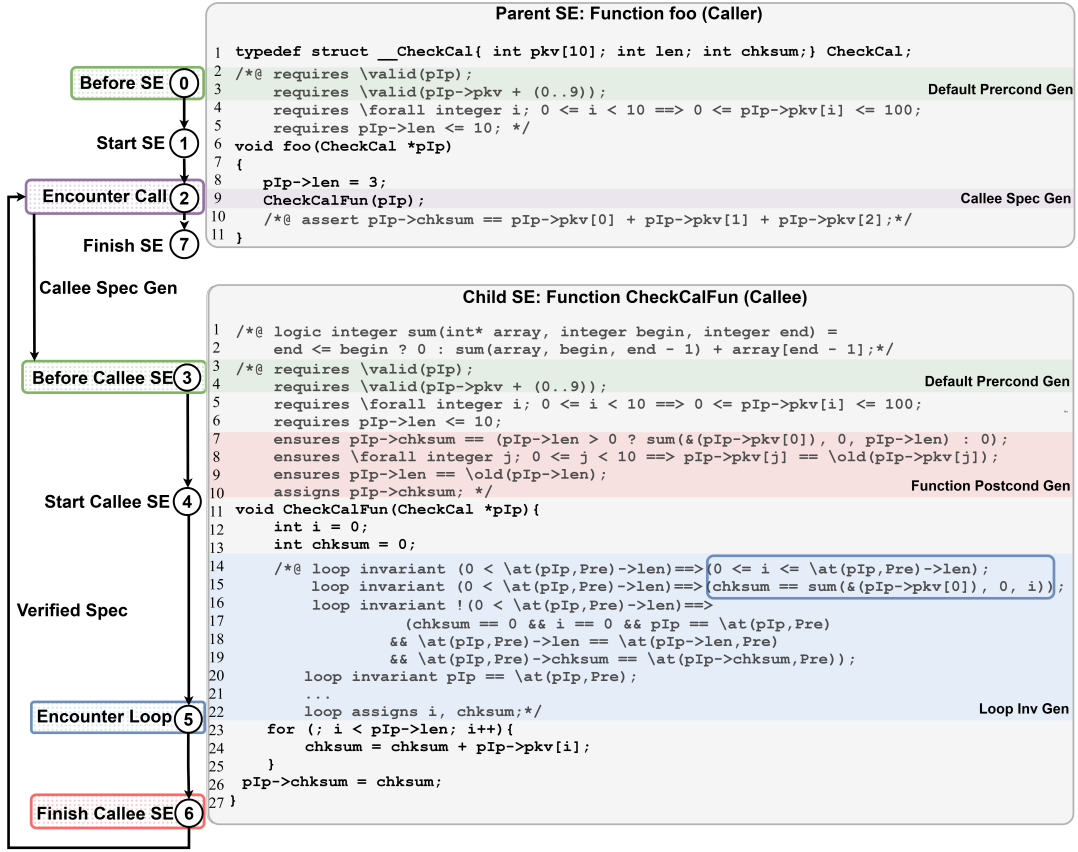


Fig. 1. Motivating Example

verification goals provided a priori, and the non-default preconditions shown on the designer-annotated `requires` lines of `CheckCalFun` in Fig. 1 reflect additional requirements from the designers.

Before presenting our method, we attempted to generate the specification for this example using GPT-4o, Claude 3.7 Sonnet, and AUTOSPEC, given both the preconditions and the verification goal. The loop invariants they produced, shown below, were all found to be incorrect:

GPT-4o

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum == \sum(0, i-1,
  \lambda integer k; pIp->pkv[k]);

```

CLAUDE 3.7 SONNET

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(integer k = 0; k < i; pIp->pkv[k]);
loop assigns i, chksum;
loop variant pIp->len - i;

```

AUTOSPEC

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum == \sum(0, i-1, pIp->pkv);
loop invariant \forall integer k;
  0 <= k < i ==> chksum >= pIp->pkv[k];
loop assigns i, chksum;

```

The failures highlight common limitations of pure LLM-based approaches. First, all three state the partial-sum relation but none guards the case in which the loop never executes (e.g., `pIp->len <= 0`): they assume loop entry, so even

the well-formed clauses fail the base case. Even when we explicitly prompted GPT-4o to “consider the case when the loop is not entered”, it still failed to produce the correct conditional invariant structure. Second, each tool invents a *different* syntax for the array aggregate—`\sum` with a lambda, with a quantified binder, or over a raw pointer—without defining it as an ACSL logic function; `\sum` is not a construct that Frama-C/WP discharges by default, so these clauses are rejected as undefined.

A. Our Approach

Fig. 2 shows an overview of our approach. Given a function and an optional verification goal, the objective is to automatically generate loop invariants and function pre-/postconditions. If a verification goal is provided, these specifications should be sufficient to prove it; otherwise, they should capture useful functional relations exposed by the available symbolic states. Following the three design choices of Section I, symbolic execution drives the pipeline and computes path-sensitive symbolic information at the current boundary; SESPEC injects these facts into invariant templates and ACSL skeletons, while a lightweight static route selects supporting examples and ACSL guidance. The LLM therefore receives both semantic facts from the current symbolic state and prompt support material matched to the program’s syntactic features. The verifier then checks the candidate, and if it fails, the parsed verifier feedback is combined with the same local symbolic

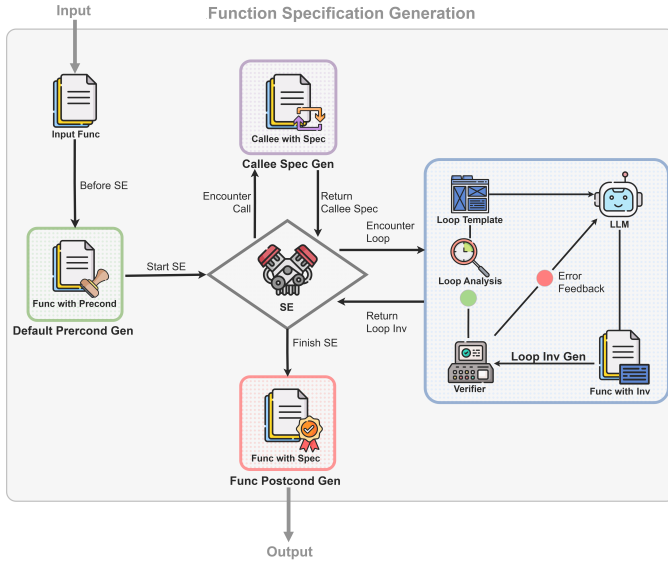


Fig. 2. Overview of Our Approach

context to drive iterative refinement until a verifier-checked summary is produced. Each generation call is issued at a symbolic-execution boundary — a loop, an unspecified callee, a recursive data structure, or a recursive function — and once a verifier-checked summary is obtained, symbolic execution resumes from the same point using that summary.

The **Function Specification Generation** process comprises four symbolic-execution-driven stages, as illustrated in Fig. 2, plus an **LLM-Fallback Layer** (Section III-D) that takes over whenever symbolic execution cannot produce a usable summary (self-recursive functions, recursive data structures, external calls, path explosion, floating-point arithmetic, or runtime errors). The four symbolic-execution-driven stages are described below. It begins with **Default Precondition Generation** (marked in green), which automatically produces memory-safety preconditions sufficient to start symbolic execution (node ① in Fig. 1); the procedure is detailed in Section III-B.

Using this precondition, the tool initiates symbolic execution from the function’s entry point (node ① in Fig. 1). When a function call is encountered (node ② in Fig. 1), the current symbolic execution is paused and **Callee Specification Generation** is invoked (marked in purple in Fig. 2). It recursively processes the callee function to generate its specification (nodes ③–⑥ in Fig. 1). Upon completion, the tool returns to the call site, incorporates the generated pre-/postcondition of the callee, and resumes symbolic execution of the caller. Throughout this process, a call stack is maintained to support nested function calls. Functions are processed in a bottom-up order, and all generated function specifications are stored for reuse across different calling contexts.

When symbolic execution encounters a loop within the current function (node ⑤ in Fig. 1), it pauses and transfers control to **Loop Invariant Generation** (marked in blue in Fig. 2). The Loop Analysis module is first invoked with the complete symbolic state at the loop entry and identifies different types of variables by analyzing the behavior of the

loop body. In the presented example, the analysis produces the following results: the unchanged variables $pIp \rightarrow len$, pIp , and $pIp \rightarrow pkv$; and the inductive variables i and $chksum$. The loop guard $i < pIp \rightarrow len$ is also recorded, together with the meta-condition that determines whether the loop body is executed at least once (here, $0 < \text{at}(pIp, Pre) \rightarrow len$). Subsequently, the tool generates parameterized invariant templates designed to capture various execution scenarios, such as whether the loop is entered, which variables remain unchanged, which variables evolve inductively, and how they behave. These templates are ACSL skeletons whose guards, unchanged-variable clauses, frame facts, and relation placeholders are derived from symbolic loop-entry and loop-body information. Below we present part of the generated templates for the motivation example (see Fig. 1 for the full set):

```
loop invariant (0 < \at(pIp, Pre) -> len)
==> PLACEHOLDER_for_i;
loop invariant (0 < \at(pIp, Pre) -> len)
==> PLACEHOLDER_for_chksum;
loop invariant pIp == \at(pIp, Pre);
```

These skeletons provide specific `PLACEHOLDER` tags for the LLM to populate (marked as the blue boxes in Fig. 1). Guided by these structured templates and symbolic execution results, the LLM produces loop invariants that capture useful relations among different types of variables. This symbolic-state-guided structure narrows the candidate space and makes key logical constraints explicit before the LLM fills the placeholders. For instance, the generated invariants correctly incorporate the loop-entry condition $(0 < \text{at}(pIp, Pre) \rightarrow len)$ — the critical logical constraint whose omission caused the pure-LLM failures in Section II.

During Loop Invariant Generation, the verifier-guided refinement procedure is iteratively invoked to assess both the validity of the generated invariants and their usefulness for proving the given verification goal when such a goal is available. Depending on verifier feedback, different refinement strategies guide the LLM toward verifier-checked invariants.

When symbolic execution of the callee is completed (node ⑥ in Fig. 1), **Function Postcondition Generation** (marked in red in Fig. 2) is triggered. The resulting symbolic states are used to synthesize a postcondition that summarizes the callee’s functional behavior. Upon completion, the generated pre-/postconditions are returned to the call site and reused as a functional summary in the caller (node ⑦ in Fig. 1), enabling symbolic execution of the caller to resume without re-analyzing the callee’s internal control flow.

This example illustrates the full pipeline; goal-independent generation is evaluated separately in Section VI.

III. PRE-/POSTCONDITION GENERATION

This section first introduces symbolic-execution-based specification generation for code segments without loops, and then presents the generation of general function specifications. Loop-invariant generation is deferred to the next section.

A. Symbolic execution and strongest postcondition

Before stating the proposition, we fix its assumptions.

We use a *flattened memory model* to explicitly track memory values and aliasing during symbolic execution. At each program point p , the *scope* is the set of variables and formal parameters accessible at p . Starting from these roots, the model enumerates the finite set of *atomic memory locations* $\mathcal{L}_p$ reachable by dereferencing pointers and selecting struct fields and array elements: a struct is flattened field by field, an array of statically known or parameter-bounded length element by element, and each resulting primitive cell (a scalar or a pointer value), is represented by a distinct typed symbolic variable. Aliasing is expressed over these location names: access paths that the model resolves to the same cell denote the same symbolic location, while pairs that can be resolved as distinct are recorded as separated, thus equality, disjointness, and per-cell update facts are all stated over $\mathcal{L}_p$.

The model produces only finite, first-order facts over named cells; it does not use recursive or inductive predicates. These facts serve as preconditions and postconditions for two backends: QCP (in QCPL) for symbolic execution, and Frama-C/WP (in ACSL) for verification. What limits the expressiveness of the facts is Frama-C/WP's requirement for automatic SMT discharge: because the verification step must be fully automatic, the facts are confined to first-order and free of recursive/inductive predicates. Hence the model is deliberately finite. It handles scalars, structs, fixed arrays, and pointers with statically resolvable aliasing, but not unbounded or recursive structures (e.g., linked lists, trees), dynamic unbounded allocation, or accesses with a symbolic offset (e.g., pointer arithmetic). These cases fall outside the exact fragment and trigger the LLM fallback described in Section III-D.

Within this model, the symbolic execution of statements maintains a *symbolic state* (PC_π, σ_π) for each execution path π . At a program point p on path π , let $\mathcal{L}_p^\pi$ denote the set of symbolic locations currently tracked by the executor. This set includes the entry-point locations $\mathcal{L}_{p_0}$ reachable from the function parameters, together with the local variables that have entered scope along π before reaching p (hence $\mathcal{L}_{p_0} \subseteq \mathcal{L}_p^\pi$).

- PC_π denotes the path condition, i.e., a logical formula over initial symbolic values and constants that encodes the conditions needed to follow the path π .
- σ_π denotes the symbolic store along path π , i.e., a mapping from each currently tracked symbolic location in $\mathcal{L}_p^\pi$ to a symbolic expression over the initial values.

The symbolic execution of a program is defined as a transformation over sets of symbolic states, branching at conditional control flow points. Formally, execution begins at the function entry point p_0 with the initial symbolic state (PC_0, σ_0) , where σ_0 assigns an initial symbolic value to each location in the entry point location set $\mathcal{L}_{p_0}$, and PC_0 is the precondition constraining those initial values. For a program point p , let the set of all symbolic states reachable from p_0 be $\{(PC_{\pi_i}, \sigma_{p, \pi_i})\}_{i=1}^m$, where π_i ranges over all paths from p_0 to p . The store $\sigma_{p, \pi}$ is a mapping, not a formula. We turn it into a formula over the tracked locations by equating each value at p with its symbolic expression:

$$\rho(\sigma_{p, \pi}) \equiv \bigwedge_{\ell \in \text{dom}(\sigma_{p, \pi})} \ell^p = \sigma_{p, \pi}(\ell).$$

Here ℓ^p is the value of ℓ at p , while the expression $\sigma_{p, \pi}(\ell)$ uses initial symbolic values. An unchanged tracked cell retains its value at the starting point; access paths known to alias refer to the same cell. Locations outside the finite tracked set are outside the exact claim. The path condition PC_π also uses initial symbolic values and includes the starting precondition. Thus the reachable state pairs at p , projected to the tracked locations, satisfy $\bigvee_{i=1}^m (PC_{\pi_i} \wedge \rho(\sigma_{p, \pi_i}))$. At entry, the store relation is the identity relation, so the formula is PC_0 together with those identity equalities.

We implement a symbolic execution procedure $\text{SEStart}(\mathcal{F}, P, p_{\text{start}})$. Given a code segment $\mathcal{F}$ and a precondition P (a formula representing symbolic states) at a starting point p_{start} , the procedure executes symbolically from p_{start} , exploring the feasible paths until it reaches the next program boundary (i.e. a loop entry, a function call, or the exit of $\mathcal{F}$), and returns the point p at which it pauses. By construction, the segment from p_{start} to p contains no loop and no function call, so its input/output behavior can be summarized exactly.

For an interval from p_{start} to p_{end} , “initial” means the state at p_{start} , even when it is not the function entry. In the proposition, $\mathcal{L}_{p_{\text{end}}}$ denotes the same observable endpoint locations tracked on every feasible path; path-local locations are omitted. The claim concerns initial values and these common endpoint locations. We first define the notion of a *strongest postcondition* in the Hoare-logic sense before stating the proposition for loop-free segments.

Definition 1. (*Strongest Postcondition*) For a segment $\mathcal{F}$ and a precondition P , a formula Φ over the tracked initial and reached state variables is the *strongest postcondition* of $\mathcal{F}$ with respect to P , if

- $\{P\} \mathcal{F} \{\Phi\}$ holds, meaning every state pair (s_0, s) reachable by executing $\mathcal{F}$ from $s_0 \models P$ satisfies Φ .
- For any Ψ over the same tracked state-pair vocabulary such that $\{P\} \mathcal{F} \{\Psi\}$ holds, we have $\Phi \Rightarrow \Psi$ as a relation over state pairs.

Let $\mathcal{F}$ be a program segment from p_{start} to p_{end} and P be a logical formula denoting the precondition. Write $\Pi = \text{Path}(p_{\text{start}}, p_{\text{end}})$ for the feasible paths explored under P , and $\Phi_{SP} = SP(\mathcal{F}, P, p_{\text{start}}, p_{\text{end}})$. We define

$$\Phi_{SP} = \bigvee_{\pi \in \Pi} (PC_\pi \wedge \rho(\sigma_{p_{\text{end}}, \pi} |_{\mathcal{L}_{p_{\text{end}}}})).$$

Then we state the following proposition.

Proposition 1. $SP(\mathcal{F}, P, p_{\text{start}}, p_{\text{end}})$ is the strongest postcondition of $\mathcal{F}$ with respect to P , if the following conditions hold:

- (A1) *Scope conditions:* $\mathcal{F}$ contains no external calls (i.e., every called function has a definition), no loops, and every memory access is representable in the flattened model. For every input satisfying P , execution of the interval is well-defined under the modeled C semantics: memory accesses, aliasing, and arithmetic operations are represented exactly and no undefined behavior occurs.
- (A2) *Symbolic execution conditions:* The symbolic execution procedure is path-complete, that is, it enumerates all feasible paths from p_{start} to p_{end} under P . Furthermore,

for each enumerated path π , the path condition PC_π exactly characterizes the inputs that cause the concrete execution to follow π , and the symbolic store $\sigma_{p_{end}, \pi}$ exactly expresses the resulting concrete state at p_{end} as a function of the initial state, according to the flattened memory semantics. These statements concern the common endpoint locations; other endpoint locations are not characterized by the formula.

The proof is given in Appendix A in the supplementary material.

If symbolic execution is unexpectedly interrupted by unsupported program features, SP returns *SymExecFailed* and the affected task is handled by the LLM-based fallback layer (Section III-D). If symbolic execution encounters undefined behavior, it reports an error instead of returning an exact strongest postcondition. The fallback may still produce a verifier-checked specification, but it no longer carries the strongest-postcondition guarantee of Proposition 1.

B. Default Precondition Generation

The default precondition generator $\text{DefaultPre}(\mathcal{F})$ generates sufficient preconditions for the supported symbolic-execution model. By default, it assumes that differently named pointers refer to disjoint memory regions and imposes configured bounds on array lengths and values. These are default generation choices, not necessary conditions for all well-defined C executions. The separation assumption does not imply that SESPEC cannot handle aliasing; it restricts the input domain of contracts containing the corresponding separation clauses.

The generator processes the function parameters: for each pointer (including those inside structs and arrays), it emits a validity fact; after all pointers are collected, it appends a single separation fact asserting that the initial memory regions of these pointers are pairwise disjoint. Bound constraints restrict array lengths and integer values to the configured analysis ranges (e.g. $0 < \text{arr_len} < \text{MAX_LEN}$, $0 \leq v \leq \text{MAX_VAL}$ for integer value v). This model is chosen because it lies at the intersection of the expressive power of QCP (our symbolic execution engine) and Frama-C/WP (our back-end verifier). The default generator does not emit additional functional assumptions such as sortedness.

Designers may supply additional precondition clauses; these are conjoined with the auto-generated precondition into PC_0 at p_0 before symbolic execution begins. Fig. 1 illustrates both modes: the validity and separation facts on `CheckCalFun` are auto-generated, while the range constraints on its array values and length field are designer supplied.

In contrast, for recursive pointer-linked data structures (linked lists, trees, recursive structs), the number of reachable memory cells is determined only at runtime (by the list length or tree depth). Therefore, the static generator cannot emit a finite set of validity/separation facts for the entire shape; instead, the shape must be captured by an inductive predicate. In this case DefaultPre delegates the generation of the inductive shape predicate to the LLM fallback layer (Section III-D),

which proposes the predicate and the corresponding precondition. The non-recursive part (validity of the head pointer and of primitive fields) is still emitted by the static generator. The LLM-generated predicate and precondition clauses are kept only if Frama-C/WP accepts them.

C. Function Specification Generation Algorithm

Algorithm 1 describes the process of generating function specifications guided by symbolic execution. The algorithm takes as inputs a function $\mathcal{F}$, a stack cS of callee functions whose specifications are not yet available, and a list aF of functions already annotated with generated specifications. It performs symbolic execution throughout the procedure, recursively invoking callee-specification generation and loop-invariant generation as needed. Upon termination, it returns an annotated version of $\mathcal{F}$ equipped with pre-/postconditions and loop invariants.

Algorithm 1: Function Specification Generation

FuncSpec

Input: function $\mathcal{F}$, call stack cS , annotated functions aF
Output: function $\mathcal{F}$ with specification S

```

1 if  $\mathcal{F} \in aF$  then
2   return
3  $cS.push(\mathcal{F})$ ;
4  $lS \leftarrow \text{InitLoopStack}()$ ;
5  $P \leftarrow \text{DefaultPre}(\mathcal{F})$ ;
6  $p_{start} \leftarrow p_0$ ;
7  $p \leftarrow \text{SEStart}(\mathcal{F}, P, p_{start})$ ;
8 while  $p \neq p_{-1}$  do
9    $P \leftarrow SP(\mathcal{F}, P, p_{start}, p)$ ;
10   $p_{start} \leftarrow p$ ;
11  if  $p = p_{loop}$  then
12     $\mathcal{F} \leftarrow \text{LoopInvGen}(\mathcal{F}, p, lS, cS, aF, P, LLM)$ ;
13  if  $p = p_{callee}$  then
14     $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$ ;
15   $p \leftarrow \text{SEStart}(\mathcal{F}, P, p_{start})$ ;
16  $S \leftarrow SP(\mathcal{F}, P, p_{start}, p_{-1})$ ;
17  $\mathcal{F} \leftarrow (\mathcal{F}, S)$ ;
18  $aF.append(cS.pop())$ ;
19 return  $\mathcal{F}$ ;
```

The algorithm works as follows. If $\mathcal{F}$ has already been annotated, it returns immediately (Lines 1–2); otherwise $\mathcal{F}$ is pushed onto the call stack cS — which records functions encountered during execution that still lack specifications — and a fresh loop stack lS is initialized for the nested loops of $\mathcal{F}$ (Lines 3–4). The default memory-safety precondition P is then computed by DefaultPre (Section III-B) at the entry point p_0 , which assigns initial symbolic values to all atomic memory locations reachable from the variables in scope at p_0 (typically the function parameters), and symbolic execution begins from p_0 via SEStart (Lines 5–7); SEStart returns the next pause point p — the next loop entry, callee site, or function exit. While p has not reached the exit point p_{-1} (Line 8), the exact postcondition holding at p is aggregated through SP over the covered loop-free interval from the symbolic-execution results and p_{start} is reset to p (Lines 9–10), after which the dispatch is by boundary kind (Lines 11–15): a loop entry triggers LoopInvGen , invoked with P as

the loop precondition and p as the loop entry, returning $\mathcal{F}$ with the loop annotated by its invariant; a callee site triggers a recursive FuncSpec on the callee to obtain its specification; and in either case symbolic execution then resumes from the updated p_{start} under the freshly produced annotations. The two stacks together implement a compositional, bottom-up traversal: callee specifications are derived first and reused across calling contexts, and nested loops are processed before their enclosing loops. When symbolic execution reaches the function exit, SP aggregates the final postcondition $\mathcal{S}$, the annotated $\mathcal{F}$ is recorded in aF , and $\mathcal{F}$ is popped from cS and returned (Lines 16–19).

Two design points are worth making explicit. First, Algorithm 1 presents the idealized symbolic-execution-driven main path. In practice, the LLM-fallback layer described in Section III-D wraps this algorithm rather than being inlined into it. The fallback handles cases where symbolic execution cannot produce a usable summary (self-recursive functions, recursive data structures, external calls, *etc.*). Second, the loop-handling counterpart LoopInvGen (Algorithm 3), invoked at Line 12, is also guided by symbolic execution. It instantiates invariant templates using the loop precondition P , the symbolic store and path conditions at the loop entry, and the unchanged and non-inductive variable sets extracted by LoopAnalysis (Section IV-B). The LLM then completes the template-specific invariant clauses, and the verifier-guided Refine step repairs them against Frama-C/WP feedback.

D. LLM Fallback

Algorithm 1 presents the idealized symbolic-execution-driven path and intentionally omits the fallback layer; the LLM-fallback layer wraps that algorithm and is triggered in exactly two cases. (i) When the target function is self-recursive or manipulates recursive data structures: the symbolic executor neither computes recursive fixpoints nor enumerates the unbounded cell set of a recursive shape, so the fallback is invoked directly at the function entry instead of running the algorithm. (ii) When $SP(\cdot)$ at the function exit returns *SymExecFailed*: this covers the residual cases in which symbolic execution cannot produce a usable summary, including external calls, path explosion, and runtime errors. Loops and ordinary callees do not enter fallback; they are handled by LoopInvGen and recursive FuncSpec respectively. Their residual verifier failures are sent to the verifier-guided refinement layer, which also wraps Algorithm 1 rather than being inlined.

In the fallback, the LLM acts as a semantic summarization component for the missing symbolic-execution step: it receives the current code fragment, available symbolic states and path conditions, previously generated callee specifications or loop summaries, and an ACSL skeleton, and proposes ACSL clause candidates for four function-level tasks — precondition generation, assigns-clause generation, postcondition generation, and inductive-predicate generation. The prompting inputs and the four tasks are detailed in Appendix B in the supplementary material.

Fallback candidates are accepted only when Frama-C/WP proves them under the configured memory model and solver

settings; verifier failures are handled by the refinement loop in Section IV-C (Table VII, Appendix F in the supplementary material). Fallback can therefore cover cases beyond the exact symbolic-execution fragment but unlike the postconditions computed by SP (Proposition 1), the results produced by the fallback are sound over-approximations verified by the prover.

IV. LOOP INVARIANT GENERATION

The general loop invariant generation problem can be defined as follows:

Definition 2. *Given a loop $l : \text{while } B \text{ do } C$ of program $\mathcal{P}$, a precondition P that holds at entry l , and an (optional) postcondition Q , the loop invariant generation problem is to find a logical formula Inv that satisfies the following conditions:*

- **Base:** $P \Rightarrow Inv$, i.e., the invariant Inv holds initially;
- **Preservation:** $\{Inv \wedge B\} C \{Inv\}$, i.e., if Inv holds at the start of a loop iteration and loop condition B is true, executing loop body C preserves Inv ;
- **Termination:** $Inv \wedge \neg B \Rightarrow Q$, i.e., upon loop termination, B turns false, Inv remains true and implies postcondition Q .

Among these conditions, the postcondition serves as the verification target, whose objective is to synthesize sufficiently strong invariants that imply it. Our invariant generation algorithm, presented below, is designed to produce strong invariants even in the absence of postconditions. We achieve this through a combination of symbolic execution and Large Language Models (LLMs), using symbolic-state-guided invariant templates that expose path conditions, variable classifications, frame facts, and relation placeholders derived from symbolic loop-entry and loop-body information. The effectiveness of this approach is experimentally demonstrated in Section V-E.

A. Invariant Generation for Loops

Algorithm 3 (Appendix C in the supplementary material) presents the invariant-generation procedure for a loop in $\mathcal{F}$. Among the inputs, p_{loop} denotes the loop entry in $\mathcal{F}$, P is the loop precondition obtained from previous symbolic execution, and the LLM serves as the synthesis component. Other parameters remain as previously defined.

The procedure separates three concerns. First (Line 1), the original outermost loop body $\mathcal{F}_{loop}$ is sent to the LLM for a chain-of-thought reading in natural language, in which the LLM is asked a few guiding questions to build a comprehensive understanding of the loop; this reading is used only as semantic support for later template filling and is never emitted as an annotation. Second (Lines 2–4), $\mathcal{F}_{loop}$ is pushed onto the loop stack and analyzed by LoopAnalysis, which symbolically executes the loop body, recursively handles all inner loops and annotates them with their invariants, and returns the sets of unchanged variables uV and non-inductive variables nV ; the outermost loop is then popped, so the stack depth is unchanged across the call. Third (Lines 5–9), the invariant for the outermost loop is synthesized: the results of LoopAnalysis feed LoopInvTemGen (Line 5), which emits the template

family for the current loop; the LLM then completes these templates through a single OuterLoopLLMCall (Line 6); and if the resulting annotation fails verification, a single call to the unified verifier-guided refinement procedure Refine (Section IV-C, Lines 8–9) refines the candidate until it passes verification.

LoopAnalysis does this by symbolically executing a single-iteration unfolding of the loop body and classifying uV and nV from the resulting postcondition; nested loops, for which no well-defined precondition exists at the enclosing loop boundary, are discharged bottom-up by direct LLM synthesis (NestedLoopLLMCall) without templates. The full procedure is given in Appendix C in the supplementary material (Algorithm 2).

B. Template Generation

The LoopInvTemGen procedure takes as inputs the loop body, the loop precondition, and the sets of unchanged and non-inductive variables, and generates invariant templates. These templates have parameters partially determined by symbolic execution results, while the remaining undetermined parameters are completed using the subsequent LLM synthesis method (i.e., OuterLoopLLMCall in Algorithm 3). The prompt support material used during instantiation, including selected examples and ACSL guidance, is constructed as described in Appendix D-B in the supplementary material. Below we introduce six forms of invariant templates. The first four are deterministically derived from symbolic execution and cover variables whose behavior is captured by SP ; the fifth is a goal-dependent template that is enabled only when a verification goal is provided at the loop boundary; the sixth is a deliberately free-form *sink* template that lets the LLM contribute invariants involving quantifiers and inductive predicates, which symbolic execution cannot anchor. Without this sink, loop invariants that rely on relations not enumerated by symbolic execution, such as quantified array properties or recursive-data-structure predicates, would have no skeleton to complete and could not be expressed.

- **Unchanged variables:** For any variable $v \in uV$ and any execution path π_j to the loop entry p , v retains its symbolic value $\sigma_{p,\pi_j}(v)$ at the loop entry:

$$PC_{\pi_j} \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- **Loop skip conditional property:** For a path π_j to p and a loop condition B , if B for path π_j is false, the loop is skipped without altering variables:

$$\neg(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- **Non-inductive variables:** For a non-inductive variable v (whose value is updated independently of its value from the previous iteration) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow (v = \sigma_{p,\pi_j}(v)) \vee \Phi(v)$$

where $\Phi(v)$ is the inductive invariant to be synthesized.

- **Inductive variables:** For an inductive variable v (which is neither unchanged nor non-inductive) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(v)$$

- **Verification goals:** In addition to variable-specific templates, we also introduce templates corresponding to each verification goal. For a verification goal g and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(g)$$

- **Free-form sink:** an unanchored slot Φ that lets the LLM propose invariants involving quantifiers and inductive predicates that the deterministic forms above cannot anchor.

This template-driven approach is crucial for our loop invariant generation algorithm. By leveraging information derived from symbolic execution, it simplifies the LLM's task, shifting from full invariant generation from scratch to completing invariant templates. It also offers structural guidance, helping the LLM produce formulas that conform to the required ACSL logical syntax. More importantly, this approach encourages strong invariants that capture relevant variable behaviors across feasible execution paths, while the final verifier-guided refinement step determines which generated clauses can be accepted by Frama-C/WP.

C. Verifier-Guided Specification Refinement

Refinement is invoked after SESPEC has produced candidate loop invariants or function specifications. A candidate can fail in three different ways: it may be syntactically incorrect (failing to conform to ACSL grammar), it may be semantically invalid (failing to be proved by Frama-C/WP), or logically weak (verifiable but too weak to prove a downstream assertion when such a target exists). The refinement procedure takes five inputs: the annotated program $\mathcal{F}$, the candidate annotation region $\mathcal{A}_0$ (the loop invariant or specification clauses to be repaired, while the rest of $\mathcal{F}$, denoted by $\mathcal{F}^-$, remains fixed), the local symbolic context Γ saved when the candidate was generated, the target kind $k \in \{loop, spec\}$, and the LLM. The output is $\mathcal{F}^*$ with a verifier-checked annotation region $\mathcal{A}^*$ when the iteration converges, or the strongest verifier-checked subset found after budget exhaustion.

Algorithm 4 (Appendix F in the supplementary material) shows the control flow. The algorithm first separates the candidate annotations from the surrounding program, storing the latter as $\mathcal{F}^-$, so every refinement round edits only the current annotation region. Each iteration runs Frama-C/WP on the current $\mathcal{F}$. If verification succeeds, the loop stops. If the verifier reports a syntax or type error before semantic proof begins, refinement follows the syntax-repair branch. Otherwise, the proof failures are classified by their proof role and the target kind k ; only the non-empty failure categories are rendered into the next prompt. If the iteration budget is exhausted, the algorithm switches to a conservative Houdini-style pruning pass.

The verifier-guided refinement stage is implemented as three layers, matching the three branches in Algorithm 4.

Layer 1: syntax repair. Layer 1 handles syntax and type errors. If Frama-C rejects the candidate before generating proof obligations, SESPEC calls the LLM with only the current annotation region and the corresponding verifier error. The

returned annotation is checked again by Frama-C/WP. This layer only makes the candidate parsable; semantic repair is handled by Layer 2.

Layer 2: strategy-driven semantic refinement or regeneration. When the candidate parses but Frama-C/WP fails to prove it, SESPEC classifies the failures into typed categories such as loop-invariant obligations, function-specification obligations, callee-instance obligations, frame obligations, and downstream assertion obligations. The complete category-to-strategy mapping is given in Table VII (Appendix F in the supplementary material). For each round, SESPEC loads only the strategy blocks whose categories are non-empty, so the prompt exposes the formal failure together with its matching fix strategy. This is the main refinement step: it revises invalid clauses, strengthens valid but insufficient clauses, or regenerates an unstable annotation region when the current candidate is globally inconsistent.

Layer 3: Houdini pruning. When the refinement budget is exhausted, SESPEC runs one Houdini-style deletion pass over the current annotation region. For loop refinement, it removes still-failing loop invariants while preserving the largest verifier-checked subset. For function-level refinement, it removes failing target `ensures` clauses and residual loop invariants. This layer is intentionally conservative: it only deletes clauses identified by verifier feedback and returns the strongest subset accepted by the verifier.

The same algorithm serves loop-level and function-level refinement; only the set of error categories produced by `Classify(Errors, k)`, and hence the strategy rows selected from Table VII differs. This keeps the control loop uniform while letting each failure category receive its own repair action. *Implementation note (best-candidate rollback).* Algorithm 4 presents the logical refinement skeleton. In the implementation, SESPEC additionally tracks the best candidate seen so far, using a lexicographic score

$$\text{Score}(\mathcal{F}) = (\mathbf{1}_{\text{parse}}, \frac{|\text{Valid}(VC)|}{|VC|}, |C|),$$

where VC is the set of verification conditions generated by Frama-C/WP for the candidate, $\text{Valid}(\cdot)$ is the subset proved, C is the number of retained clauses, and $\mathbf{1}_{\text{parse}}$ is 1 iff the candidate parses. Candidates are ordered lexicographically by this score, preferring parseability, then proof ratio, then the number of retained clauses. If two consecutive rounds do not improve the best score (*patience* = 2), the implementation restores the best recorded candidate and resumes refinement from that snapshot. If the iteration budget is exhausted, Houdini starts from the same best candidate.

V. EXPERIMENTS

This section evaluates SESPEC, our symbolic execution-based approach to automatic specification generation. The main experiments use benchmark families from prior invariant and specification generation work, which enables direct C/ACSL comparisons under established settings. Section VI complements this evaluation with a separate goal-independent study, where assertions are removed and the generator must decide what functional behavior to specify.

A. Experimental Setup

Benchmark: We evaluate our approach on benchmark families commonly used by recent specification and invariant generation tools. These include: **SyGuS** [20], collected by CODE2INV [10]; **OOPSLA** [21]; **NLA**, collected by LI-PuS [11]; the datasets from **SV-COMP** and **Frama-C**, collected by AUTOSPEC [13]. In addition, we constructed three new benchmarks to capture more program scenarios: The **pIp** set consists of small C fragments adapted from a navigation component of an industrial control codebase, involving pointer and struct manipulations as well as branching structures. We use **pIp** as a small, self-contained industrial sample rather than as a claim about the complexity or scale of the original system. We carefully adapt and reformulate **pIp** programs into a format acceptable to Frama-C and QCP, thereby establishing a new dataset, which constitutes one of our contributions; the **list-S** set is derived from the LIG-MM [22] benchmark and includes all singly linked list programs from SLING and SV-COMP; and the **numer-S** set is a curated collection of additional programs, comprising 20 linear and 20 nonlinear loops, primarily drawn from CLAUSE2INV [12] and harder than those in **SyGuS** and **NLA**.

Verifier: All experiments are performed using Frama-C version 27.1 (Cobalt). We employ the WP plugin as the verification backend, which enforces a 10-second timeout per query, uses Z3 as the primary SMT solver, and applies the Typed memory model.

Model: We evaluate four LLMs of increasing capability: **GPT-3.5-turbo**, **GPT-4o-mini**, **GPT-4o**, and **Claude 3.7 Sonnet**. For all experiments, we use a fixed sampling temperature of 0.7 to balance output diversity and stability.

Below we evaluate our approach through four research questions:

- **RQ1:** Which program classes can SESPEC handle?
- **RQ2:** What syntactic correctness, validity, and accuracy does SESPEC achieve?
- **RQ3:** How much do symbolic execution and refinement each contribute?
- **RQ4:** Does SESPEC outperform LLM- and SMT-based baselines?

B. RQ1: Which program classes can SESPEC handle?

Table I summarizes the 395 benchmark programs in our evaluation. They span linear, non-linear, nested/multi-loops, function calls, pointer manipulations, and recursive data types such as linked lists. The suite includes both loop-only programs (blue cells, invariant generation only) and programs requiring full specifications (red cells, loop invariants plus pre/postconditions).

SESPEC verified 353 of 395 programs (89.4%) across numerical loops, pointer/struct fragments, linked lists, and **pIp** fragments adapted from an industrial navigation control codebase. The core symbolic execution algorithm does not systematically handle arbitrary recursion or external library/system calls; the LLM fallback covers some simple recursive cases (see Appendix I in the supplementary material), and broader support is left to future work. These failures reflect programs

TABLE I
DATASET OVERVIEW BY PROGRAM TYPE.

Program Type	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S	#Program	#Solved
Linear Loop	133	30	14	0	20	7	0	0	204	196
Non Linear Loop	0	0	0	30	20	3	0	0	53	41
Nested & Multi Loop	0	16	7	0	0	1	0	0	24	15
Memory & Loop	0	0	2	0	0	17	12	24	53	45
Iteration Free	0	0	0	0	0	20	38	0	58	56
External & Recursive Call	0	0	0	0	0	3	0	0	3	0
#Program	133	46	21	30	40	51	50	24	395	
#Solved	132	40	16	25	34	39	44	23		353
Avg. LoC	19	24	22	21	19	18	57	24		

that are difficult for both the LLM and the SMT backend. For nested and multi-loop programs, symbolic execution often cannot obtain precise loop preconditions, so the LLM must infer relations across loop phases or loops. For non-linear programs, the missing facts often require non-linear arithmetic reasoning, and in some cases helper functions or inductive definitions that SMT solvers cannot prove within the budget. Recursive data structures similarly require inductive predicates beyond the finite flattened-memory summary. External and recursive calls fail for a different reason: they fall outside the symbolic-execution fragment, leaving no exact symbolic anchor for synthesis.

Lines of code do not reliably reflect difficulty on these benchmarks — the binding constraint is invariant complexity. SESPEC reaches 96% on Linear and Iteration-Free programs but drops to 77% on Non-Linear loops and 63% on Nested/Multi-Loops, where invariants must capture richer numerical relations, data-structure properties, and inter-loop interactions. The 0% on External & Recursive Call (3 programs) is the largest residual gap and is consistent with the scope discussion in Appendix J in the supplementary material.

Finding (RQ1). SESPEC produces a *Valid* specification for 89.4% (353/395) of programs spanning linear/non-linear loops, memory, and linked lists; the residual failures concentrate on reasoning-heavy cases involving nested/multi-loop relations, non-linear arithmetic, recursive data structures, or external/recursive calls outside the symbolic-execution fragment.

C. RQ2: What syntactic correctness, validity, and accuracy does SESPEC achieve?

We evaluate correctness through three metrics:

- **Syntax rate (Syn.):** the percentage of generated specifications that are syntactically accepted by the ACSL/Frama-C front end;
- **Validity rate (Val.):** the percentage of generated specifications that are syntactically correct and whose generated clauses are accepted by Frama-C/WP under the verifier configuration in Section V. For benchmarks that contain pre-existing assertions or verification targets, those targets are excluded from Val.; Val. measures whether the generated specification itself can be verified. For example, generated loop invariants are valid only if they satisfy both the *Base* and *Preservation* conditions;

TABLE II
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1 AND PASS@5. THE COMPLETE PASS@1/3/5 TABLE IS GIVEN IN TABLE VIII (APPENDIX G IN THE SUPPLEMENTARY MATERIAL).

Benchmark	Model	Pass@1 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.67	63.04	97.83	93.57	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	64.71	50.98	49.02
	GPT-3.5-turbo	77.08	60.42	52.08	83.33	62.50	54.17
	GPT-4o	82.35	72.55	66.67	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	87.50	65.00	92.50	92.50	75.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83

- **Accuracy rate (Acc.):** the percentage of generated specifications that are syntactically correct, valid, and strong enough to prove the benchmark verification goal. For pre/postconditions, this means that they imply the given target assertions (e.g., a functional property); for loop invariants, it further requires the *Termination* condition.

These metrics are assessed under Pass@k: a benchmark is counted as successful if any of up to k complete generation attempts satisfies the reported metric. Each attempt is a com-

plete pipeline run and may include syntax repair, verification, the internal refinement loop, and pruning; refinement iterations are not counted as additional Pass@k attempts. The Pass@1 cost measurements for the goal-independent study are reported separately in Section VI-D.

Table II presents a comprehensive overview of our experimental results.

The observed gaps among different evaluation metrics reveal important characteristics of both benchmark difficulty and model capability. As benchmark complexity increases, the discrepancies between Syn., Val., and Acc. become more pronounced. A similar trend is observed across models: weaker models tend to exhibit larger gaps between these metrics. The starkest instance is GPT-3.5-turbo on **list-S**, where 70.83% syntactic correctness collapses to 20.83% validity and only 8.33% accuracy at Pass@1 — the model emits well-formed linked-list annotations that the verifier rejects — whereas Claude 3.7 Sonnet on the same benchmark reaches 95.83%/83.33%/75.00%. Notably, Val. does not trivially coincide with Syn. in our results, indicating that the generated specifications are not simplistic invariants (e.g., `true`) or trivial properties that programs would satisfy by default. Instead, our approach aims to capture non-trivial functional relations beyond syntax-valid or vacuous annotations, which naturally induces a meaningful distinction between syntactic correctness and semantic validity.

Importantly, our method demonstrates consistent effectiveness across different models and reduces the performance gap between stronger and weaker ones in several benchmark families. For example, on benchmarks such as **SV-COMP**, **NLA**, and **numer-S**, GPT-4o and Claude achieve identical performance at higher Pass@k levels (Pass@3 and Pass@5), suggesting that symbolic guidance can improve the usefulness of comparatively weaker models. Furthermore, the method exhibits rapid convergence under Pass@k evaluation. Performance improvements from Pass@3 to Pass@5 are marginal across several benchmarks, including **NLA**, **numer-S**, and **pIp**.

The results also indicate comparatively lower accuracy on the **OOPSLA**, **SV-COMP**, and **Frama-C** benchmarks. For **OOPSLA** and **SV-COMP**, this is primarily due to the presence of nested and multiple loops, where symbolic execution often fails to derive sufficiently strong preconditions. In addition, the absence of explicit loop conditions in **OOPSLA** further complicates invariant inference. In the case of **Frama-C**, the difficulty mainly stems from complex loop structures involving array manipulations, external function calls, and recursive behaviors, which remain challenging for symbolic execution. Despite these obstacles, our method still achieves notable performance on these challenging benchmarks. Moreover, the improvements from Pass@1 to Pass@5 show that additional attempts can help recover from difficult generation cases under the evaluated budget.

Finding (RQ2). Syntactic correctness is generally high, but the gap between validity and accuracy widens with benchmark complexity and narrows with model capability; SESPEC narrows model gaps on several benchmark families and reaches most Pass@k gains by Pass@3, with most residual accuracy loss on nested/multi-loop and memory-heavy families.

D. RQ3: How much do symbolic execution and refinement each contribute?

To assess the individual contributions of each component, we conducted an ablation study on 100 programs randomly sampled from the benchmark families in Table I, following that table’s split into 70 loop-only programs (invariant generation only) and 30 programs requiring full specifications (function-level cases). For a fair comparison under controlled compute, all configurations are matched on LLM token usage and the refinement loop is capped at 3 iterations. This token-matched ablation isolates each component on the goal-dependent benchmarks; the complementary ablation in Section VI re-measures the same components on the 400-program goal-independent benchmark without a compute cap.

We compare three configurations: **Only LLM (Pass@3)** uses the LLM alone for invariant generation without external guidance; **LLM + symbolic execution (Pass@3)** adds symbolic-execution-derived templates and program context; and **LLM + symbolic execution + refinement (Pass@1)** adds the verifier-guided refinement loop on top.

TABLE III
ABLATION STUDY.

Method	Syn. (%)	Val. (%)	Acc. (%)
<i>Loop invariant generation (70 programs)</i>			
Only LLM	97.50	37.50	28.75
LLM + sym. exec.	95.71	62.86	60.00
LLM + sym. exec. + refine	95.71	95.71	87.21
<i>Program specification generation (30 programs)</i>			
Only LLM	66.67	26.67	16.67
LLM + sym. exec.	76.67	50.00	46.67
LLM + sym. exec. + refine	93.33	86.67	86.67

The ablation in Table III shows that the LLM alone yields high syntactic correctness but low validity and accuracy. Adding symbolic execution lifts validity and accuracy substantially in both settings (average +28 percentage points, abbreviated pp) while reducing syntactic correctness only marginally on the loop-only setting (−1.79 pp). Adding the refinement loop pushes loop-only validity to 95.71% and lifts function-level validity from 50.00% to 86.67%, demonstrating that each component contributes a distinct and complementary gain. The largest individual jump comes from the refinement loop, which converts symbolic-execution-derived candidates that the verifier rejects into verifier-checked ones; this is consistent with the cost analysis in Section VI, where refinement is also the dominant cost driver.

TABLE IV
ACCURACY COMPARISON ACROSS BENCHMARKS AND BASELINES. SESPEC REPORTS PASS@5 (CF. TABLE II); BASELINES USE THEIR BEST REPORTED CONFIGURATION.

Method	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S
CODE2INV	54.9%	—	—	—	—	—	—	—
HOLA	—	93.5%	—	—	—	—	—	—
CLN2INV	93.2%	—	—	—	—	—	—	—
LIPuS	93.2%	—	—	83.3%	—	—	—	—
CLAUSE2INV	99.2%	—	—	93.3%	—	—	—	—
AUTOSPEC-3.5	85.7%	82.6%	76.2%	70.0%	75.0%	60.8%	14.0%	0.0%
AUTOSPEC-4o	86.5%	71.7%	66.7%	70.0%	72.5%	68.6%	22.0%	0.0%
AUTOSPEC-Claude 3.7	88.7%	80.4%	66.7%	60.0%	55.0%	74.5%	4.0%	0.0%
ACInv-3.5	71.4%	45.7%	42.9%	—	—	34.5%	—	—
ACInv-4o	79.9%	76.1%	76.2%	—	—	48.3%	—	—
SESPEC-3.5	93.2%	52.2%	66.7%	56.7%	67.5%	51.0%	76.0%	16.7%
SESPEC-4o	99.2%	76.1%	76.2%	86.7%	82.5%	72.6%	78.0%	79.2%
SESPEC-Claude 3.7	100.0%	87.0%	76.2%	86.7%	85.0%	76.5%	88.0%	95.8%

Finding (RQ3). Symbolic-execution guidance and verifier-guided refinement contribute distinct, complementary gains: over the LLM alone, symbolic execution adds ~ 28 pp of validity/accuracy, and refinement then lifts validity to 95.7% on loop invariants and to 86.7% on full specifications.

E. RQ4: Does SESPEC outperform LLM- and SMT-based baselines?

Table IV is an effectiveness comparison under each method’s reported configuration, not a compute-matched comparison. SESPEC uses at most five complete generation attempts per program. The original AUTOSPEC study repeated each program five times, with iterative synthesis and verification within each run [13]. ACInv also uses an internal augmentation loop; its published analysis reports settings of 1, 3, 5, and 10 augmentation cycles [14]. These counts do not equalize LLM calls, tokens, or verifier work across methods, and the numerical synthesis baselines do not have a directly comparable LLM-sampling budget.

Overall comparison with LLM-based approaches. Table IV compares AUTOSPEC, ACInv, and SESPEC; SPEC GEN is omitted here because it targets Java/JML rather than C/ACSL. On the four datasets originally introduced by AUTOSPEC, SESPEC achieves higher accuracy than ACInv on both GPT-3.5-turbo and GPT-4o, and both tools show similar gains from upgrading the model.

In contrast, AUTOSPEC barely improves from GPT-3.5-turbo to GPT-4o. Its reliance on few-shot prompting plus exhaustive sampling and verification-based filtering caps the gain that stronger models can deliver, so our AUTOSPEC-4o reproduction stays close to the reported AUTOSPEC-3.5 numbers.

SESPEC instead gains steadily with stronger models. Its workflow requires complex reasoning — program-execution simulation plus verifier-guided invariant refinement — which exceeds GPT-3.5-turbo’s instruction-following limits. Symbolic execution narrows this reasoning surface, so stronger models like GPT-4o benefit more.

The **pIp** fragments make this concrete: AUTOSPEC with Claude 3.7 Sonnet performs poorly despite Claude’s strong

reasoning, because Claude struggles to emit syntactically correct ACSL, and a specification that does not parse is never verified. SESPEC avoids this with an ACSL skeleton built from the symbolic state, which both fixes the surface syntax and pins down the symbolic facts each clause must state; its advantage here is therefore semantic as well as syntactic.

Detailed results across benchmarks. Against SMT-based numerical baselines, SESPEC is on par with CLAUSE2INV on **SyGuS**, exceeds LIPuS on the nonlinear **NLA** benchmark while trailing CLAUSE2INV by 6.6 pp, and achieves 87% on the harder nested/multi-loop **OOPSLA** dataset. These SMT-based tools are specialized for numerical loops.

Against the LLM baselines AUTOSPEC and ACInv on numerical benchmarks, SESPEC scores 15% higher on **SyGuS** and higher on **NLA** and **numer-S**; on **OOPSLA** it matches ACInv and is only marginally below AUTOSPEC (with Claude 3.7 Sonnet, SESPEC is 5% higher). All three tools perform equally on **SV-COMP**, likely because the dataset is dominated by nested/multi-loop cases that no current approach solves. Overall, symbolic guidance captures specification facts that pure neural baselines miss on these benchmarks.

On **Frama-C**, SESPEC improves over AUTOSPEC via inductive-predicate support, which enables more expressive specifications. On the industrial **pIp** fragments (structs and pointer operations), the flattened memory model plus symbolic execution gives an average 50% higher success rate than AUTOSPEC. On the linked-list **list-S** benchmark, SESPEC also performs well; AUTOSPEC cannot handle this class because it lacks inductive-data-type support.

Finding (RQ4). SESPEC matches tools specialized for numerical invariants and clearly outperforms LLM-based generators (AUTOSPEC, ACInv), with the largest margins on the pointer-, struct-, and linked-list families that prior LLM tools handle poorly; its accuracy also scales with model capability where AUTOSPEC’s plateaus.

VI. GOAL-INDEPENDENT EVALUATION

The previous section evaluates SESPEC under established benchmark protocols used by prior invariant and specification-generation tools. Many of those programs include explicit

`assert` statements, which are useful for prior tools to generate specifications strong enough to discharge those assertions, i.e., verification goals, but they do not fully exercise the goal-independent setting SESPEC is designed to support. To complement that evaluation, we run a separate study on goal-independent programs, where all `assert` statements are removed and the generator must decide what functional behavior to specify. For this comparison we select the state-of-the-art specification-generation tool in each of the two language ecosystems we span: AUTOSPEC [13] for C/ACSL and SPECGEN [15] for Java/JML.

A. Experimental Setup

We construct **SESPEC-400**, a 400-program C benchmark built from three specification-generation suites: **FormalBench** [23] (250 programs) and **SpecGenBench** (59), translated to C from Java, and the native-C **SV-COMP** (91); programs are stripped of `assert` statements so the generator must decide what to specify. The **FormalBench** base dataset contains Java programs verified in OpenJML with sequential, branching, and loop-based structures. The translation preserves the source-level control flow and data dependencies used by the benchmark programs (Appendix E in the supplementary material). Verification follows the Frama-C/WP protocol of Section V (Z3, Typed memory model).

All LLM-based methods are evaluated on `gpt-4o`, `gpt-5-mini`, `gpt-5.4-mini`, and `gpt-5` at sampling temperature 0.7 under Pass@1. We deliberately draw all four from a single model family: models from different families differ too much in training data and capability for a clean comparison, whereas staying within one family lets performance differences reflect reasoning capability rather than cross-family data bias. This set also differs from the models in Section V, which were fixed by the baselines compared there.

We compare four C/ACSL workflows: full SESPEC (up to 10 refinement iterations), SESPEC *w/o refine*, SESPEC *w/o refine w/o symbolic execution (Pure LLM)*, and AUTOSPEC in its default one-shot configuration; plus SPECGEN, which is run on the original Java versions in its native Java/JML pipeline. The two ablations, *Pure LLM* and *w/o refine*, isolate the contributions of symbolic-execution guidance and verifier-guided refinement by comparison with the full pipeline, while AUTOSPEC serves as the verifier-level baseline that shares our C/ACSL setting. SPECGEN is the closest goal-independent system but runs in the Java/JML ecosystem, so we include it only as a cross-ecosystem reference rather than a direct verifier-level baseline. Each run is capped at a 1800-second outer timeout.

This study, run on all 400 **SESPEC-400** programs without the per-program compute cap of Section V, probes the goal-independent setting along four orthogonal axes: whether the pipeline produces *valid* specifications, whether those specifications are *strong*, at what *cost*, and how it *fails*. Throughout, we say a program is *valid* when the generated specification is accepted in full by the verifier, i.e., *all* verification conditions it induces are discharged within the configured budget; a specification with any unproved obligation is not counted, so

TABLE V
NATIVE-VERIFIER VALIDITY ON SESPEC-400.

Method	Model	Valid	Valid (%)
AUTOSPEC (C/ACSL)	<code>gpt-4o</code>	115	28.8
	<code>gpt-5-mini</code>	104	26.0
	<code>gpt-5.4-mini</code>	255	63.8
	<code>gpt-5</code>	288	72.0
SPECGEN (Java/JML)	<code>gpt-4o</code>	144	36.0
	<code>gpt-5-mini</code>	161	40.2
	<code>gpt-5.4-mini</code>	198	49.5
	<code>gpt-5</code>	251	62.7
SESPEC w/o refine w/o symbolic execution (Pure LLM)	<code>gpt-4o</code>	52	13.0
	<code>gpt-5-mini</code>	76	19.0
	<code>gpt-5.4-mini</code>	106	26.5
	<code>gpt-5</code>	55	13.8
SESPEC w/o refine	<code>gpt-4o</code>	126	31.5
	<code>gpt-5-mini</code>	107	26.8
	<code>gpt-5.4-mini</code>	132	33.0
	<code>gpt-5</code>	144	36.0
SESPEC	<code>gpt-4o</code>	272	68.0
	<code>gpt-5-mini</code>	333	83.3
	<code>gpt-5.4-mini</code>	373	93.3
	<code>gpt-5</code>	372	93.0

validity here measures whole-specification verification rather than partial proof. Extending the four research questions of Section V, this study is organized around four further questions, one per axis:

- **RQ5:** How often does SESPEC produce a *Valid* specification?
- **RQ6:** How strong are SESPEC’s specifications versus the baselines?
- **RQ7:** What does SESPEC cost in time, tokens, and API calls?
- **RQ8:** What are SESPEC’s dominant failure modes?

B. RQ5: How often does SESPEC produce a Valid specification?

Table V reports the per-model results. Within the C/ACSL workflows, the full SESPEC pipeline achieves the highest *Valid* rate on every model: 68.0% on `gpt-4o`, 83.3% on `gpt-5-mini`, 93.3% on `gpt-5.4-mini`, and 93.0% on `gpt-5`. These correspond to absolute gains of +55.0, +64.3, +66.8, and +79.2 pp over Pure LLM, and +39.2, +57.3, +29.5, and +21.0 pp over AUTOSPEC. The ablations show two different sources of improvement. Refinement is the dominant source of validity gain, but it is also the most expensive component: enabling refinement lifts *Valid* by +36.5, +56.5, +60.3, and +57.0 pp across the four models, while requiring repeated LLM calls and verifier-feedback rounds. This confirms that iterative repair is highly effective, but its gain comes with substantial LLM-iteration cost.

By contrast, symbolic-execution guidance provides a more cost-effective source of improvement. Even without refinement, adding symbolic-execution information raises *Valid* over Pure LLM on every model: from 13.0% to 31.5% on `gpt-4o`, 19.0% to 26.8% on `gpt-5-mini`, 26.5% to 33.0% on `gpt-5.4-mini`, and 13.8% to 36.0% on `gpt-5`, whose unusually low

13.8% baseline is syntactic rather than semantic—it favors advanced ACSL constructs (behaviors, axiomatics) but often violates their surface syntax, triggering fatal whole-file parse errors that the symbolic-execution context helps avoid. Unlike refinement, this gain does not rely on repeated LLM sampling; it comes from exposing the local symbolic context before synthesis. Moreover, for loop-free programs fully supported by the flattened memory model, symbolic execution can directly compute the path-sensitive postcondition, so the core functional summary can be produced without an LLM call.

The refinement-budget ablation on `gpt-5.4-mini` (Appendix F in the supplementary material, Fig. 7) shows that the gain from refinement saturates after $k \approx 5$ iterations, that the final Houdini pass contributes a further +14.1 pp, and that the differential refinement prompts with best-candidate rollback together account for roughly 23 pp of additional validity over a plain error-plus-code repair prompt.

Finding (RQ5). On **SESPEC-400** the full pipeline reaches 68–93% *Valid*; refinement is the dominant but most expensive source of gain, whereas symbolic-execution guidance is a cheaper one that needs no repeated LLM sampling.

C. RQ6: How strong are SESPEC’s specifications versus the baselines?

Validity certifies that a specification is discharged by its verifier, but it does not rank how informative it is. We therefore compare accepted input domains (preconditions), normal-return guarantees (postconditions), and predicates at corresponding loop points (loop invariants) independently.

LLM judge and expert review. Each dimension uses a separate judge prompt and API call. Preconditions are classified by set inclusion between their accepted input domains; postconditions are compared by bidirectional implication over the common input domain; and invariants are compared at corresponding loop points. Thus, the two directional segments in Fig. 3 mean more permissive for preconditions and logically stronger for postconditions and invariants. We do not aggregate them into an overall contract ranking. A missing precondition or postcondition denotes `true`. Expert review corrected 32 judge verdicts, shown as transitions in Table IX (Appendix H in the supplementary material) for both baselines; any implication that cannot be established is included in *Incomp*. Appendix H in the supplementary material also gives the judge prompts and reports the configuration, reviewer background, and A/B-order audit.

Fig. 3 shows that SESPEC leads AUTOSPEC on postconditions and loop invariants. AUTOSPEC repeatedly samples the LLM and accumulates verifier-accepted `ensures` and invariant clauses, including many redundant, tautological, or trivially implied facts. SESPEC instead supplies the local symbolic state at each generation boundary, directing the model to generate path-sensitive relations and inductive predicates while reducing redundant clauses.

In the primary ordering, AUTOSPEC more often declares broader accepted input domains, frequently through a missing

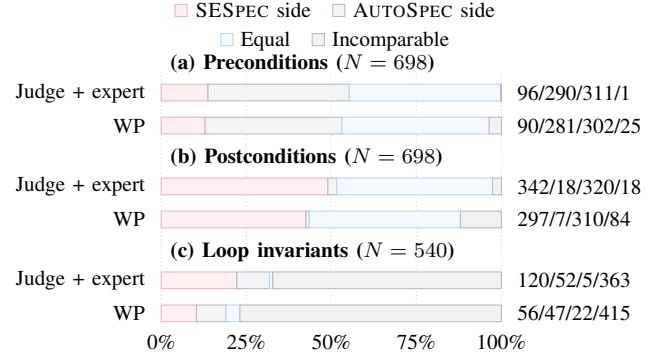


Fig. 3. Dimension-wise relations for AUTOSPEC versus SESPEC over identical data sets. A directional segment means a more permissive accepted domain for preconditions and a logically stronger predicate for postconditions or invariants. “Judge + expert” is the corrected judge result; WP checks the same implications. Right-hand counts follow the legend order.

or `true` precondition, whereas SESPEC’s default preconditions explicitly record runtime- and memory-safety requirements. We report this as a difference in declared permissiveness rather than an intrinsic quality advantage: a broader declared domain is meaningful only when the additional inputs are valid executions for which the contract remains meaningful.

The cross-language strength comparison against SPECGEN (Java/JML) is deferred to Appendix H in the supplementary material, because no single verifier stack applies across the two ecosystems; we treat it as contextual evidence rather than as a controlled comparison of the generation algorithms.

Frama-C/WP cross-check. We cross-check the same complete AUTOSPEC–SESPEC sets with separate Frama-C/WP harnesses: 698 pairs for preconditions and postconditions and 540 for invariants. With Frama-C 27.1, Alt-Ergo and Z3, and a five-second per-goal limit, the SESPEC/baseline/equal/incomparable counts are 90/281/302/25, 297/7/310/84, and 56/47/22/415, respectively. A direction fails unless WP proves every obligation; unsupported harnesses and unresolved loop correspondence are also included in *Incomp*. WP cannot distinguish a genuinely false implication from a true implication whose proof is incomplete; both situations are recorded as *Incomp*. Formal implication checking is constrained by ATP timeouts and automation limits, which leave substantially more pairs incomparable, especially for loop invariants. The LLM judge followed by expert review can inspect the semantic relations in these unresolved cases and therefore yields a more complete and fine-grained classification, while WP serves as an independent formal cross-check. Fig. 3 confirms the precondition and postcondition trends, while WP orders fewer invariants.

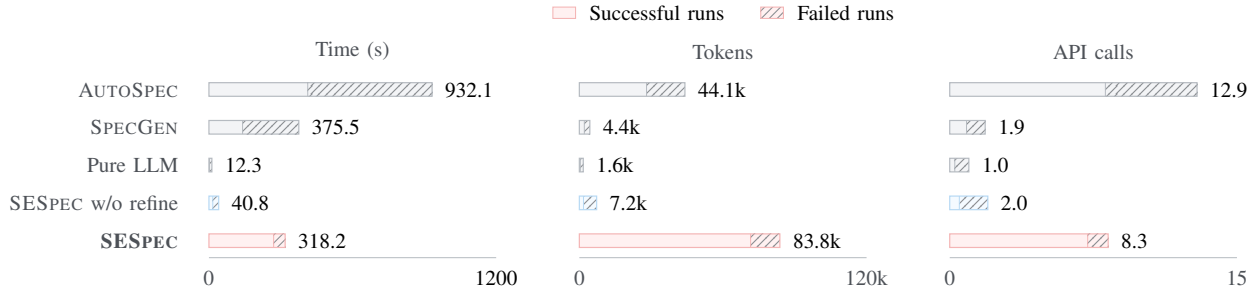


Fig. 4. Mean cost per attempted program on all 400 gpt-5.4-mini cases. Each bar stacks valid-run (solid) and failed-run (hatched) contributions, each divided by 400.

Finding (RQ6). Symbolic context changes where specification strength appears: SESPEC yields more informative postconditions and stronger loop invariants than AUTOSPEC in the shared C/ACSL setting, while AUTOSPEC’s broader declared input domains mainly reflect safety assumptions that are not written explicitly. The cross-language comparison against SPECGEN (Java/JML) is reported in Appendix H in the supplementary material; no single total ordering applies across all dimensions and languages.

D. RQ7: What does SESPEC cost in time, tokens, API calls, and memory?

This cost analysis is scoped to the **SESPEC-400** study in Section VI; it is not collected over the established benchmark families in Section V. We use this scope because **SESPEC-400** is the goal-independent setting in which SESPEC, its ablations, AUTOSPEC, and SPECGEN are run under the same Pass@1 protocol. The paired comparison in Fig. 4 covers all 400 programs on gpt-5.4-mini and reports cost per attempted program, including failures. The original AUTOSPEC time, API-call count, and validity label are retained. Memory use is negligible: the process never exceeds ~ 500 MB peak RSS across the evaluated benchmarks.

Each solid segment in Fig. 4 sums the cost of valid runs and divides it by all 400 attempts; the hatched segment uses the same denominator for failed runs. Failures account for 519.4 of AUTOSPEC’s 932.1 seconds and 235.0 of SPECGEN’s 375.5 seconds per attempt, compared with 48.1 of SESPEC’s 318.2 seconds. Dividing total time by valid-run count gives 338.6 seconds per obtained valid SESPEC specification, versus 1,462.1 for AUTOSPEC and 758.6 for SPECGEN on this cohort. SESPEC’s 83.8k tokens per attempt include repeated refinement context; token counts apply no cache discount.

Finding (RQ7). SESPEC’s added framework complexity mainly appears as higher token use; its wall-clock time and API-call count are much closer to the baselines than the raw token totals alone suggest.

E. RQ8: What are SESPEC’s dominant failure modes?

To separate terminal outcomes from the errors seen in the detailed logs, Fig. 5 reports two views of an independent gpt-5.4-mini rerun on **SESPEC-400**, executed to capture per-case failure logs. This rerun yields 321 valid cases, fewer

than the 373 (93.3%) of the main run in Table V; the gap is intentional, as we cap this rerun at 3 refinement iterations (versus up to 10 in the main run) precisely to expose more failure cases for characterization. The left panel gives the final case-level outcome, while the right panel classifies the 79 non-valid cases (the 42 *No valid* plus 37 *No syntax* cases) by the dominant source program feature observed in the input and failure logs. Table XIII (Appendix J in the supplementary material) separately counts logged syntax events and prover failures by outcome (*Timeout*, *Unknown*, *Failed*) and by obligation content (*Loop Inv.*, *Post Cond.*, *Assigns*, and other verification conditions). The categories in Fig. 5 are mutually exclusive for counting; if several features occur in one program, we assign the case to the first feature that explains the blocking obligation in the log.

The event distribution in Table XIII shows where failed runs accumulate. Timeouts are dominated by loop invariant obligations ($2,590/4,370 = 59.3\%$), especially preservation and establishment goals over arrays or heap-backed arrays. Postcondition and *assigns* timeouts are smaller but still substantial, indicating that the hard cases are not confined to one prompt or clause type.

The inspected logs contain *Timeout*, *Unknown*, and *Failed* outcomes, corresponding to proof timeouts, prover uncertainty, and backend or resource-level execution failures. These outcomes are not semantic counterexamples and cannot, by themselves, distinguish an incorrect generated specification from a correct but difficult proof obligation that the current verification backend does not discharge. In our manual review of the programs, generated specifications, and unresolved obligations, we more often observed candidate properties that were broadly consistent with the program but difficult for the current backend to prove, particularly for complex loop, array, and heap reasoning. This qualitative assessment does not rule out specification-synthesis errors in individual cases. The method and verifier boundaries exposed by these observations are discussed in Appendix J in the supplementary material.

Finding (RQ8). Residual failures manifest predominantly as SMT timeouts on complex proof obligations. The logs cannot conclusively separate specification-synthesis errors from proof obligations beyond the current backend, although manual inspection more often points to verifier limitations.

Complementing this quantitative study, Appendix I in the

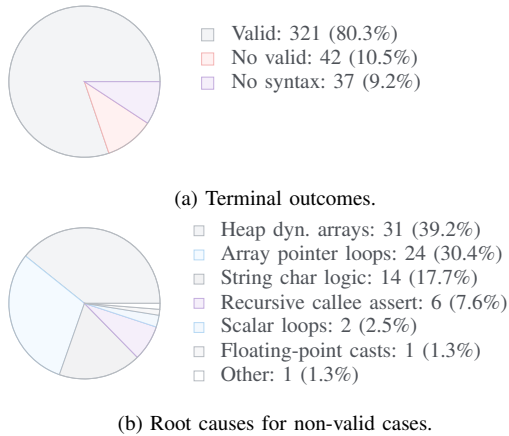


Fig. 5. Failure outcome and root-cause summary for the gpt-5.4-mini SESPEC-400 rerun.

supplementary material presents ten case studies: five principal cases cover phase-split loop invariants from weak goals, path-complete postconditions under C aliasing, shape-predicate routing for linked structures, the contrast between RTE-safety and functional specification, and the scale boundary on a 225-line industrial control pipeline (13 of 14 functions specified; 601 of 625 verification conditions discharged, 96.2%), and five supplementary cases probe inductive-helper synthesis and the recursion and SMT-timeout boundaries.

VII. RELATED WORK

Traditional approach. Traditional techniques for program specification and invariant inference can be broadly categorized into dynamic and static methods. Dynamic approaches infer specifications and invariants by analyzing runtime data collected during test execution. Daikon [9] executes test cases to gather program traces and produces a set of candidate invariants based on a predefined library of templates, which are subsequently filtered using the observed data. Sharma et al. [24] employ a similar approach to generate candidate invariants, but additionally verify their soundness using SMT solvers. In contrast, DIG [25] bypasses template-based filtering and instead directly infers invariants from execution traces by integrating techniques from equation solving, polyhedra construction, and theorem proving. Despite their practicality, dynamic methods are highly dependent on the coverage and quality of the test suites.

Static methods eliminate the dependency on test cases and explore a variety of techniques, including constraint solving [1], [2], abstract interpretation [26] [27], abductive inference [21], Craig interpolation [3]–[5], recurrence analysis [28]–[30], shape analysis, and other methods [31] [32]. Representative lines include Farkas’-lemma-based constraint solving with quantifier elimination [1], [2], convex-polyhedra abstract interpretation with widening [27], interpolation-based synthesis from reachability analyses [3], [5], recurrence-based methods that require closed-form solutions [28]–[30], and semidefinite-programming synthesis for polynomial templates [33]. These static methods are often not scalable, may require manual intervention, and are typically specialized for

programs adhering to specific forms (e.g., certain types of loops or invariants); our approach leverages LLMs to cover a broader set of benchmark patterns in the evaluated C/ACSL setting.

Machine learning and SMT-based approach. Machine learning techniques and Large Language Models have been increasingly used for synthesizing loop invariants. Recent advancements [10], [11], [34]–[36], particularly in deep learning and reinforcement learning, have spurred new approaches for invariant and specification generation. CODE2INV [10], based on Counterexample-Guided Inductive Synthesis (CEGIS) [37], uses graph neural networks to learn neural representations of program structures. While CODE2INV employs a reinforcement learning approach, it often faces challenges with limited performance and sparse rewards. To address these issues, LIPuS [11] establishes a clearer division of labor between the capabilities of RL and SMT solvers and introduces a novel two-dimensional reward mechanism to alleviate the sparse reward problem of CODE2INV. CLN2INV [38] learns loop invariants with continuous logic networks. CLAUSE2INV [12] refines the guess-and-check paradigm into a generate-combine-check framework, employing an LLM to generate candidate clauses and a counterexample-driven combiner to synthesize them into loop invariants. Loopy [39] uses LLM-generated ACSL invariant candidates, Houdini-style filtering, and error-guided repair for C loop benchmarks checked by Frama-C. However, CLN2INV, LIPuS, and CLAUSE2INV are mostly confined to numeric programs and do not constitute end-to-end functional-specification generators; instead, they produce invariants in SMT format. Loopy moves to ACSL loop annotations, but it remains goal-driven loop-invariant inference rather than goal-independent functional specification generation. Moreover, these tools are goal-dependent, i.e., they rely on predefined verification goals to generate accurate specifications. Empirical numbers against CLN2INV, LIPuS, and CLAUSE2INV on SyGuS / NLA / OOPSLA appear in Table IV (Section V); SESPEC is on par with CLAUSE2INV on SyGuS and falls between LIPuS and CLAUSE2INV on the nonlinear NLA benchmark, while these numerical tools do not report numbers on the harder nested/multi-loop OOPSLA dataset that SESPEC covers.

LLM-based approach. Pei *et al.* [40] demonstrate that fine-tuning pre-trained LLMs on code can effectively predict program invariants, with better performance than Daikon when the set of available tests is limited. However, like Daikon, their approach does not provide formal guarantees for the generated invariants. SPECGEN [15] is a Java-oriented tool that employs a two-phase process to generate specifications: it first leverages an LLM to produce invariant candidates, and then applies mutation operators with a weighted heuristic strategy to generate verifiable specifications when initial generation fails. Preguss [16] adopts a divide-and-conquer strategy that leverages potential RTE assertions from abstract interpretation-based static analyzers to guide LLM-based synthesis of minimal specifications, particularly preconditions that represent the weakest constraints sufficient to guarantee RTE-freeness. AUTOSPEC [13] decomposes programs into smaller parts and

TABLE VI
POSITIONING OF SESPEC AGAINST REPRESENTATIVE SPECIFICATION AND INVARIANT GENERATION TOOLS.

Tool	Target	Goal	Method	Program Support	Array/Struct/Rec.	Validation
Daikon	invariants	test-driven	traces/templates	Java/C/C++ traces	support/no opt.	trace filter
LIPuS	loop inv.	goal-driven	SMT/RL clauses	C loops	unsupported	SMT
CLN2INV	loop inv.	goal-driven	continuous logic nets	C loops	unsupported	SMT
CLAUSE2INV	loop inv.	goal-driven	LLM clauses + CEX	C loops	unsupported	SMT
Loopy	loop inv.	goal-driven	LLM + Houdini repair	C loops	unsupported	Frama-C
AUTOSPEC	ACSL specs	goal-driven	decomposition + few-shot	C function/loops	support/no opt.	Frama-C
ACInv	ACSL specs	goal-driven	static abstractions	C function/loops	optimized	Frama-C
Preguss	ACSL specs	goal-driven	RTE alarms	C function/loops	support/no opt.	Frama-C
SPECGEN	JML specs	goal-independent	LLM + mutation	Java function/loops	support/no opt.	OpenJML
SESPEC	ACSL specs	goal-independent	symbolic states/templates	C function/loops	optimized	Frama-C

uses carefully designed prompts including few-shot examples to guide an LLM in generating specifications, which are then verified using Frama-C. ACInv [14] uses static analysis to extract essential information about loops and data structures, which is then embedded into prompts for an LLM to generate candidate invariants and abstract predicates. We provide a detailed comparison in Section V with ACInv [14] and AUTOSPEC [13], both of which target C programs.

The closest conceptual neighbor to SESPEC is ACInv, because both approaches inject program-analysis information into prompts. The main difference is the source and precision of that information. ACInv uses static loop and data-structure abstractions, which are useful for describing local program shape but can lose path-dependent equalities, must-alias versus may-alias distinctions, and correlations across struct fields or pointer dereferences. SESPEC instead mines symbolic-state snapshots. These snapshots preserve the path conditions under which a relation holds, the concrete memory updates induced by each path, and separation constraints required by ACSL verification. This is why the template source in SESPEC is symbolic rather than fixed, dynamic, or purely static.

Preguss and SPECGEN are related but address different settings. Preguss synthesizes minimal specifications for runtime-error freedom from abstract-interpretation-generated RTE targets; this makes it goal-dependent and safety-oriented rather than a functional summarization method. In SESPEC, potential RTE reports on the Frama-C validation side are recorded as warnings rather than as the primary success criterion: the pipeline uses QCP symbolic execution to expose functional behavior, may stop and fall back when an RTE prevents symbolic execution, and asks the refinement LLM to strengthen preconditions when doing so helps avoid such failures.

Lean-based specification generation. A separate line of recent work pursues specification synthesis and end-to-end formal verification inside the Lean proof assistant. CLEVER [41] and VERINA [42] curate algorithm-level benchmarks for which the specification, the implementation, and the proof are all written in Lean, exploiting Lean’s dependent type theory and expressive tactic language to encode higher-order invariants, inductive predicates, and proof obligations that are awkward to express in ACSL or JML. These efforts demonstrate that, given enough expressive power on the logic side, LLMs can drive specification and proof synthesis for non-trivial algorithmic programs. The fundamental restriction,

however, is that the implementation language is forced to be Lean itself: target programs must be written (or rewritten) in Lean’s functional, dependently-typed syntax, which rules out specifying and verifying legacy C, Java, or other production codebases without a full reimplementation. SESPEC targets the opposite trade-off — it gives up some of Lean’s expressive richness in exchange for working directly over off-the-shelf C source, so that specifications can be attached to the implementation language in which industrial software is actually written.

Across all four lines, no prior system targets *goal-independent, verifier-checked functional* specifications for off-the-shelf C: invariant-centric tools including LIPuS, CLN2INV, CLAUSE2INV, and Loopy are goal-driven and largely numeric-loop oriented; Loopy moves to ACSL/Frama-C but still synthesizes loop invariants from verification goals; AUTOSPEC and ACInv still depend on a goal and inject only static or few-shot context; SPECGEN removes the goal but in the Java/JML ecosystem; and Lean-based tools require reimplementing the program in Lean. SESPEC occupies this gap by driving generation with path-sensitive symbolic state over native C source (Table VI).

VIII. CONCLUSION

We presented SESPEC, an end-to-end framework for generating verifier-checked C program specifications by grounding LLM synthesis in symbolic states and verifier feedback.

The experiments also expose the current boundary of this approach: as analyzed in Appendices J and K in the supplementary material, residual failures manifest predominantly as timeouts on inductive, recursive, and heap-heavy proof obligations under the current Frama-C/WP backend. The logs do not fully distinguish synthesis errors from correct but difficult obligations, so further progress may require both improved synthesis and a wider verification surface.

Looking ahead, specification generation may become more agentic as reasoning models improve. Instead of following a fixed symbolic-execution-driven order, future systems could let an LLM agent decide when to invoke symbolic execution, strengthen specifications or introduce lemmas. The key principle should remain unchanged: every generated specification should be grounded in symbolic states and verifier feedback, rather than free-form generation.

DATA AVAILABILITY

All datasets, implementation code, experimental results, and supplementary materials related to this work are publicly available at <https://github.com/taylor-swift-13/SESPEC>.

REFERENCES

- [1] M. Colón, S. Sankaranarayanan, and H. Sipma, “Linear invariant generation using non-linear constraint solving,” in *Computer Aided Verification CAV*, ser. Lecture Notes in Computer Science, vol. 2725. Springer, 2003, pp. 420–432.
- [2] H. Liu, H. Fu, Z. Yu, J. Song, and G. Li, “Scalable linear invariant generation with farkas’ lemma,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 204–232, 2022.
- [3] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanán, and H. Hansen, “FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers,” in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 793–803.
- [4] T. Gan, L. Dai, B. Xia, N. Zhan, D. Kapur, and M. Chen, “Interpolant synthesis for quadratic polynomial inequalities and combination with EUF,” in *Automated Reasoning - 8th International Joint Conference, IJCAR*, ser. Lecture Notes in Computer Science, vol. 9706. Springer, 2016, pp. 195–212.
- [5] T. Gan, B. Xia, B. Xue, N. Zhan, and L. Dai, “Nonlinear Craig Interpolant Generation,” in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 415–438.
- [6] D. Beyer, M. Kettl, and T. Lemberger, “Decomposing software verification using distributed summary synthesis,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1307–1329, 2024.
- [7] D. Wonisch and H. Wehrheim, “Predicate analysis with block-abstraction memoization,” in *International Conference on Formal Engineering Methods*. Springer, 2012, pp. 332–347.
- [8] C. Calcagno, D. Distefano, P. O’Hearn, and H. Yang, “Compositional shape analysis by means of bi-abduction,” in *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 2009, pp. 289–300.
- [9] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Sci. Comput. Program.*, vol. 69, no. 1, pp. 35–45, 2007, special issue on Experimental Software and Toolkits.
- [10] X. Si, A. Naik, H. Dai, M. Naik, and L. Song, “Code2Inv: A Deep Learning Framework for Program Verification,” in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 151–164.
- [11] S. Yu, T. Wang, and J. Wang, “Loop Invariant Inference through SMT Solving Enhanced Reinforcement Learning,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. Association for Computing Machinery, 2023, p. 175–187.
- [12] W. Cao, G. Wu, T. Xu, Y. Yao, H. Wei, T. Chen, and X. Ma, “Clause2inv: A generate-combine-check framework for loop invariant inference,” *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1009–1030, 2025.
- [13] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, and C. Tian, “Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification,” in *Computer Aided Verification: 36th International Conference, CAV 2024, Montreal*. Springer-Verlag, 2024, p. 302–328.
- [14] R. Liu, G. Li, M. Chen, L.-I. Wu, and J. Ke, “Enhancing Automated Loop Invariant Generation for Complex Programs with Large Language Models,” *CoRR*, vol. abs/2412.10483, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2412.10483>
- [15] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, “SpecGen: Automated Generation of Formal Program Specifications via Large Language Models,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, May 2025, pp. 666–666.
- [16] Z. Wang, T. Lin, M. Chen, H. Li, M. Yang, X. Yi, S. Qin, Y. Luo, X. Li, B. Gu, L. Lu, and J. Yin, “A tale of 1001 loc: Potential runtime error-guided specification synthesis for verifying large-scale programs,” *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1874–1902, Apr. 2026. [Online]. Available: <http://dx.doi.org/10.1145/3798268>
- [17] L. Zhou, J. Qin, Q. Wang, A. W. Appel, and Q. Cao, “VST-A: A foundationally sound annotation verifier,” *Proc. ACM Program. Lang.*, vol. 8, no. POPL, 2024.
- [18] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, N. Zhan, Z. Hu, and Q. Cao, “Qcp: A practical separation logic-based c program verification tool,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.12878>
- [19] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, “Frama-C: A software analysis perspective,” in *International conference on software engineering and formal methods*. Springer, 2012, pp. 233–247.
- [20] R. Alur, D. Fisman, S. Padhi, R. Singh, and A. Udupa, “Sygus-comp 2018: Results and analysis,” *CoRR*, vol. abs/1904.07146, 2019.
- [21] I. Dillig, T. Dillig, B. Li, and K. McMillan, “Inductive invariant generation via abductive inference,” in *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, ser. OOPSLA ’13. Association for Computing Machinery, 2013, p. 443–456.
- [22] C. Liu, X. Wu, Y. Feng, Q. Cao, and J. Yan, “Towards general loop invariant generation: a benchmark of programs with memory manipulation,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 129 120–129 145, 2024.
- [23] T. Le-Cong, B. Le, and T. Murray, “Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 21 991–22 014.
- [24] R. Sharma, S. Gupta, B. Hariharan, A. Aiken, P. Liang, and A. V. Nori, “A data driven approach for algebraic loop invariants,” in *Programming Languages and Systems*. Springer Berlin Heidelberg, 2013, pp. 574–592.
- [25] T. Nguyen, D. Kapur, W. Weimer, and S. Forrest, “DIG: A Dynamic Invariant Generator for Polynomial and Array Invariants,” *ACM Trans. Softw. Eng. Methodol.*, vol. 23, no. 4, pp. 1–30, 2014.
- [26] P. Cousot and R. Cousot, “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fix-points,” in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL ’77. Association for Computing Machinery, 1977, p. 238–252.
- [27] P. Cousot and N. Halbwachs, “Automatic discovery of linear restraints among variables of a program,” in *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL ’78. Association for Computing Machinery, 1978, p. 84–96.
- [28] E. Rodríguez-Carbonell and D. Kapur, “Automatic generation of polynomial loop invariants: Algebraic foundations,” in *International symposium on Symbolic and algebraic computation (ISSAC)*. Springer, 2004, pp. 266–273.
- [29] —, “Generating all polynomial invariants in simple loops,” *J. Symb. Comput.*, vol. 42, no. 4, pp. 443–476, 2007.
- [30] J. Cyphert and Z. Kincaid, “Solvable polynomial ideals: The ideal reflection for program analysis,” *Proc. ACM Program. Lang.*, vol. 8, no. POPL, pp. 724–752, 2024.
- [31] M. Chalupa and J. Strejček, “Backward Symbolic Execution with Loop Folding,” in *Static Analysis*. Springer International Publishing, 2021, pp. 49–76.
- [32] T. Nguyen, M. B. Dwyer, and W. Visser, “SymInfer: inferring program invariants using symbolic states,” in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE ’17. IEEE Press, 2017, p. 804–814.
- [33] H. Wu, Q. Wang, B. Xue, N. Zhan, L. Zhi, and Z. Yang, “Synthesizing Invariants for Polynomial Programs by Semidefinite Programming,” *ACM Trans. Program. Lang. Syst.*, vol. 47, no. 1, pp. 1 – 35, Feb. 2025.
- [34] S. Chakraborty, S. Lahiri, S. Fakhoury, A. Lal, M. Musuvathi, A. Rastogi, A. Senthilnathan, R. Sharma, and N. Swamy, “Ranking LLM-generated loop invariants for program verification,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 9164–9175. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.614/>
- [35] H. Wu, C. Barrett, and N. Narodytska, “Lemur: Integrating Large Language Models in Automated Program Verification,” in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=Q3YaCghZNt>
- [36] M. A. A. Pirzada, G. Reger, A. Bhayat, and L. C. Cordeiro, “LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE ’24. Association for Computing Machinery, 2024, p. 1395–1407.
- [37] A. Abate, C. David, P. Kesseli, D. Kroening, and E. Polgreen, “Counterexample Guided Inductive Synthesis Modulo Theories,” in *Computer*

- Aided Verification*. Springer International Publishing, 2018, pp. 270–288.
- [38] G. Ryan, J. Wong, J. Yao, R. Gu, and S. Jana, “CLN2INV: learning loop invariants with continuous logic networks,” in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.
 - [39] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S. K. Lahiri, A. Lal, A. Rastogi, S. Roy, and R. Sharma, “Finding inductive loop invariants using large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2311.07948>
 - [40] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, “Can large language models reason about program invariants?” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23. JMLR.org, 2023, pp. 27 496 – 27 520.
 - [41] A. Thakur, J. Lee, G. Tsoukalas *et al.*, “CLEVER: A curated benchmark for formally verified code generation,” in *Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025.
 - [42] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song, “VERINA: Benchmarking verifiable code generation,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=0A4Uf88pog>